

TUGAS BESAR IF-25-21013 STRATEGI ALGORITMA

SEMESTER GENAP TAHUN 2026/2027



Disusun Oleh:

Kelompok: TibaTibaJadiKelompok

Mochamad Rasya Khairiza (124140002)

Maulana Muhamad Rizki (124140008)

Krisna Dwi Cristian (124140152)

Program Studi Teknik Informatika

Fakultas Teknologi Industri

Institut Teknologi Sumatera

2026

DAFTAR ISI

DAFTAR ISI.....	1
BAB 1: Deskripsi Tugas.....	3
1.1 Latar Belakang.....	3
1.2 Spesifikasi Tugas.....	3
1.2.1 Spesifikasi Wajib.....	3
1.2.1 Spesifikasi Bonus.....	4
BAB 2: Landasan Teori.....	5
2.1 Algoritma Greedy.....	5
2.2 Elemen-Elemen Algoritma Greedy.....	5
2.3 Cara Kerja Bot Robocode.....	6
2.4 Implementasi Algoritma Greedy pada Bot.....	6
2.5 Komponen Teknis Tank.....	7
BAB 3: Aplikasi Strategi Greedy.....	8
3.1 Mapping Persoalan Robocode ke Elemen Greedy.....	8
3.1.1 Himpunan Kandidat.....	8
3.1.2 Himpunan Solusi.....	8
3.1.3 Fungsi Seleksi.....	9
3.1.4 Fungsi Kelayakan.....	9
3.1.5 Fungsi Solusi.....	9
3.1.6 Fungsi Objektif.....	9
3.2 Eksplorasi 4 Alternatif Solusi Greedy.....	10
3.2.1 Main Bot - TibaTibaJadiKelompok (Heuristic: Akurasi Tertinggi).....	10
3.2.2 Bot Alt 1 - AggressiveHunter (Heuristic: Target Energi Terendah).....	10
3.2.3 Bot Alt 2 - ClosestTarget (Heuristic: Target Energi Terendah).....	11
3.2.4 Bot Alt 3 - EnergyManager (Heuristic: Firepower Optimal).....	11
3.3 Analisis Efisiensi dan Efektivitas.....	12
3.4 Strategi Greedy yang Dipilih.....	12
BAB 4: Implementasi dan Pengujian.....	13
4.1 Implementasi Bot Utama - Main Bot.....	13
4.1.1 Pseudocode.....	13
4.1.2 Struktur Data dan Fungsi.....	14
4.2 Pengujian.....	15
4.2.1 Pengujian 1 - Main Bot vs RamFire.....	15
4.2.2 Pengujian 2 - Main Bot vs Fire.....	15
4.2.3 Pengujian 3 - Main Bot vs Walls.....	15
4.3 Analisis Hasil Pengujian.....	16
BAB 5: Kesimpulan dan saran.....	17
5.1 Kesimpulan.....	17
5.2 Saran.....	17

LAMPIRAN.....	18
DAFTAR PUSTAKA.....	19

BAB 1: Deskripsi Tugas

Tugas Besar mata kuliah IF25-21013 Strategi Algoritma ini bertujuan untuk mengimplementasikan algoritma Greedy dalam pembuatan bot permainan Robocode Tank Royale. Robocode Tank Royale adalah permainan pemrograman di mana pemain membuat kode bot berupa tank virtual untuk berkompetisi melawan bot lain di arena pertempuran.

1.1 Latar Belakang

Robocode adalah permainan pemrograman yang bertujuan membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale. Nama Robocode merupakan singkatan dari bot Code yang berasal dari versi asli permainan ini. Robocode Tank Royale adalah evolusi dari permainan ini, di mana bot dapat berpartisipasi melalui internet atau jaringan.

Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas membuat program yang menentukan logika atau otak bot. Program yang dibuat berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjata, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

1.2 Spesifikasi Tugas

1.2.1 Spesifikasi Wajib

- Buatlah 4 bot (1 utama dan 3 alternatif, **tetap dikumpulkan 1 bot yang terbaik saja**) dalam bahasa C# (.net) yang mengimplementasikan *algoritma Greedy* pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota **minimal 2 orang** dan **maksimal 3 orang**.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi *greedy* yang diimplementasikan **harus berbeda** untuk setiap bot yang diimplementasikan dan setiap strategi greedy harus menggunakan **heuristic** yang berbeda.
- Bot yang dibuat **TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH**. Baik dari starter pack maupun dari repository engine asli.
- Buatlah strategi *greedy* terbaik, karena setiap **bot utama** dari masing-masing kelompok akan diadu dalam kompetisi Tubes.

- Strategi *greedy* yang kelompok anda buat harus **dijelaskan dan ditulis secara eksplisit** pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain **harus dapat dijalankan pada game engine** yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan **kode sumber yang dibuat**. Artinya semua strategi harus diimplementasikan
- Mahasiswa disarankan membaca dokumentasi dari *game engine*. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini **SUDAH DIMODIFIKASI**. Untuk Perubahan dapat dilihat pada bagian starter pack.

1.2.1 Spesifikasi Bonus

- (maks 10) Membuat video tentang aplikasi *greedy* pada bot serta simulasinya pada game kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. **Semakin menarik video, maka semakin banyak poin yang diberikan.**
- (maks 10) Beberapa kelompok pemenang lomba kompetisi akan mendapatkan nilai tambahan berdasarkan posisi yang diraih.

BAB 2: Landasan Teori

2.1 Algoritma Greedy

Algoritma Greedy adalah sebuah pendekatan untuk memecahkan permasalahan yang beroperasi dengan memilih opsi terbaik di setiap tahapan berdasarkan situasi yang ada (pilihan optimal lokal), dengan harapan pilihan ini akan menghasilkan solusi terbaik secara keseluruhan (solusi optimal global) (Cormen et al., 2009).

Ciri-ciri utama dari algoritma Greedy adalah bahwa setiap keputusan yang diambil tidak akan direvisi kembali (tanpa pengembalian). Karena itu, metode ini biasanya memiliki tingkat kompleksitas yang lebih rendah dan waktu eksekusi yang lebih cepat jika dibandingkan dengan strategi pencarian menyeluruh seperti brute force atau dynamic programming (Munir, 2020).

Dalam permainan Robocode Tank Royale, penerapan algoritma Greedy bisa digunakan untuk membuat keputusan secara real-time, seperti menentukan waktu yang tepat untuk menembak, memilih target, arah pergerakan, dan besaran kekuatan tembakan yang akan digunakan. Bot akan selalu melakukan tindakan yang dianggap paling menguntungkan pada situasi itu tanpa perlu melakukan perhitungan jangka panjang yang rumit.

2.2 Elemen-Elemen Algoritma Greedy

Elemen	Penjelasan
Himpunan Kandidat (C)	Kumpulan semua kemungkinan pilihan yang dapat diambil pada setiap langkah
Himpunan Solusi (S)	Himpunan kandidat yang telah dipilih dan membentuk solusi
Fungsi Seleksi	Fungsi yang memilih kandidat terbaik dari himpunan kandidat yang belum dipilih
Fungsi Kelayakan	Fungsi yang memeriksa apakah suatu kandidat dapat ditambahkan ke himpunan solusi tanpa melanggar constraint
Fungsi Tujuan (Objektif)	Fungsi yang mengukur nilai dari solusi yang dihasilkan, yang ingin dimaksimalkan atau diminimalkan

Fungsi Solusi	Fungsi yang memeriksa apakah himpunan solusi sudah lengkap
---------------	--

Konsep elemen-elemen pada algoritma Greedy digunakan untuk menentukan proses pengambilan keputusan secara lokal pada setiap langkah penyelesaian masalah (Munir, 2020).

2.3 Cara Kerja Bot Robocode

Robocode Tank Royale adalah sebuah permainan yang melibatkan pemrograman bot tank di mana para pemain menulis kode untuk bot yang bertanding secara otomatis di dalam sebuah arena (Nelson, 2001). Bot beroperasi sebagai program eksternal yang terhubung dengan mesin permainan melalui WebSocket di port yang telah ditentukan.

Setiap bot mempunyai loop utama yang terus berjalan selama pertandingan. Dalam loop ini, bot dapat melakukan berbagai tindakan, seperti bergerak, memutar tubuh tank, menggerakkan radar, dan menembakkan peluru. Semua tindakan dari setiap bot akan diproses secara bersamaan oleh mesin permainan di setiap giliran permainan.

Informasi mengenai lawan diperoleh melalui radar. Ketika radar berhasil mendeteksi lawan, mesin permainan akan memicu event `OnScannedBot()` dan mengirimkan informasi seperti posisi lawan, arah pergerakan, jarak, dan energi lawan kepada bot. Selain itu, terdapat pula event lain seperti `OnHitWall()`, `OnHitBot()`, dan `OnHitByBullet()` yang memungkinkan bot untuk merespons terhadap situasi tertentu selama pertandingan.

Sistem event ini memungkinkan bot untuk membuat keputusan secara dinamis dan dalam waktu nyata berdasarkan kondisi arena pada saat itu.

2.4 Implementasi Algoritma Greedy pada Bot

Implementasi algoritma Greedy pada bot dilakukan dengan cara membuat pilihan terbaik setiap kali terjadi peristiwa tertentu, terutama ketika musuh terdeteksi oleh radar melalui peristiwa `OnScannedBot()`.

Pada bot utama yang dirancang, heuristik yang diutamakan adalah akurasi tembakan. bot hanya akan melepaskan tembakan jika sudut antara senjata dan musuh memiliki perbedaan maksimal 3 derajat. Pendekatan ini dipilih karena kemungkinan peluru mengenai sasaran jauh lebih tinggi dibandingkan dengan menembak secara acak.

Di samping itu, bot menerapkan strategi wall hugging, yaitu bergerak sepanjang tepi arena secara konstan. Strategi ini bertujuan untuk mengecilkan sudut serangan lawan sehingga meningkatkan kemungkinan bertahan hidup.

Setiap keputusan diambil secara langsung berdasarkan situasi saat itu tanpa melakukan pencarian solusi secara global atau proyeksi jangka panjang yang rumit. Pendekatan ini sejalan dengan prinsip algoritma Greedy yang mengutamakan solusi lokal terbaik pada setiap langkah.(Cormen et al., 2009).

2.5 Komponen Teknis Tank

Tank dalam Robocode Tank Royale terdiri dari 3 bagian yang dapat bergerak secara independen:

- Body: Bagian utama tank untuk bergerak. Kecepatan putar maksimal 10 derajat per turn, bergantung pada kecepatan bot.
- Gun (Turret): Digunakan untuk menembak peluru. Kecepatan putar maksimal 20 derajat per turn.
- Radar: Digunakan untuk memindai posisi musuh dalam radius 1200 piksel. Kecepatan putar maksimal 45 derajat per turn.

BAB 3: Aplikasi Strategi Greedy

3.1 Mapping Persoalan Robocode ke Elemen Greedy

Elemen	Implementasi dalam Robocode
Himpunan Kandidat (C)	Semua bot musuh yang terdeteksi oleh radar setiap turn, beserta semua kemungkinan aksi: arah gerak, besar firepower (0.1-3.0), dan arah tembakan
Himpunan Solusi (S)	Kumpulan aksi yang dipilih setiap turn: perintah gerak, rotasi body/gun/radar, dan perintah tembak
Fungsi Seleksi	Kumpulan aksi yang dipilih setiap turn: perintah gerak, rotasi body/gun/radar, dan perintah tembak
Fungsi Kelayakan	Aksi valid jika: energi bot > 0 , gun tidak sedang panas ($\text{GunHeat} == 0$), dan bot tidak berada di luar batas arena
Fungsi Tujuan (Objektif)	Memaksimalkan total skor akhir: Bullet Damage + Bullet Bonus + Survival Score + Last Survival Bonus + Ram Damage + Ram Bonus
Fungsi Solusi	Pertempuran selesai ketika hanya tersisa 1 bot atau batas turn tercapai

3.1.1 Himpunan Kandidat

Himpunan kandidat adalah kumpulan semua musuh yang saat ini terdeteksi oleh radar bot. Setiap kali radar mendeteksi musuh melalui event `OnScannedBot`, bot memperoleh data musuh tersebut berupa posisi (X, Y), energi, kecepatan, dan arah gerak. Berbeda dengan pendekatan yang menyimpan memori musuh dalam dictionary, bot `TibaTibaJadiKelompok` memproses kandidat secara langsung per event — setiap musuh yang terdeteksi langsung dievaluasi sebagai kandidat untuk ditembak pada turn tersebut.

3.1.2 Himpunan Solusi

Himpunan solusi dalam implementasi bot ini adalah keputusan tembak yang diambil pada setiap turn: berapa besar firepower yang digunakan dan apakah kondisi sudah memenuhi syarat untuk menembak. Keputusan ini bersifat greedy bot tidak menyimpan rencana jangka panjang, melainkan mengevaluasi kondisi saat ini dan langsung mengeksekusi tembakan terbaik yang tersedia jika syarat terpenuhi.

3.1.3 Fungsi Seleksi

Fungsi seleksi adalah inti dari heuristik bot TibaTibaJadiKelompok. Fungsi ini mengevaluasi satu kondisi kritis: seberapa besar sudut antara arah gun saat ini dengan posisi musuh (gunBearing). Bot hanya akan menembak jika nilai absolut gunBearing tidak melebihi 3 derajat. Rumusan fungsi seleksi adalah:

$$\text{gunBearing} = \text{GunBearingTo}(\text{musuh.X}, \text{musuh.Y})$$

Tembak jika: $|\text{gunBearing}| \leq 3 \text{ derajat AND GunHeat} == 0$

Threshold 3 derajat dipilih karena pada jarak pertempuran tipikal (150-600 piksel), kesalahan sudut 3 derajat masih menghasilkan hit probability yang sangat tinggi. Semakin kecil gunBearing, semakin besar firepower yang diizinkan melalui formula $\text{fp} = \min(3 - |\text{gunBearing}|, \text{Energy} - 0.1)$.

3.1.4 Fungsi Kelayakan

Tidak setiap kondisi layak untuk menembak. Fungsi kelayakan pada bot TibaTibaJadiKelompok memeriksa dua syarat minimum: (1) GunHeat harus bernilai 0, artinya gun sudah dingin dan siap menembak kembali — menembak saat gun masih panas tidak akan menghasilkan peluru; (2) Energi bot harus lebih dari 0.1 agar bot tidak menghabiskan sisa energi terakhirnya untuk menembak, yang bisa membuat bot langsung dinonaktifkan jika terkena serangan setelahnya.

3.1.5 Fungsi Solusi

Fungsi solusi menentukan kapan solusi sudah lengkap dan siap dieksekusi. Dalam bot TibaTibaJadiKelompok, solusi dianggap valid dan siap dieksekusi apabila ketiga kondisi terpenuhi secara bersamaan: $\text{gunBearing} \leq 3 \text{ derajat}$, $\text{GunHeat} == 0$, dan $\text{Energy} > 0.1$. Jika salah satu kondisi tidak terpenuhi, bot tidak menembak pada turn tersebut dan melanjutkan pergerakan wall hugging sambil menunggu kondisi yang lebih baik.

3.1.6 Fungsi Objektif

Fungsi objektif merepresentasikan tujuan akhir yang ingin dioptimalkan: memaksimalkan total skor pertempuran. Secara spesifik, bot TibaTibaJadiKelompok mengoptimalkan dua komponen skor utama: (1) Bullet Damage dengan hanya menembak saat akurasi sangat tinggi ($\text{bearing} < 3^\circ$), setiap peluru hampir pasti mengenai target, memaksimalkan akumulasi Bullet Damage dan Bullet Damage Bonus; (2) Survival Score dengan bergerak terus menyusuri pinggir arena, bot meminimalkan kemungkinan tertembak, sehingga dapat bertahan lebih lama dan mengumpulkan Survival Score (50 poin per bot yang mati saat bot masih hidup) serta Last Survival Bonus.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

3.2.1 Main Bot - TibaTibaJadiKelompok (Heuristic: Akurasi Tertinggi)

Strategi: Bot selalu memilih target dengan energi paling rendah di antara semua musuh yang terdeteksi, karena musuh dengan energi rendah paling mudah dibunuh sehingga menghasilkan Bullet Damage Bonus sebesar 20%.

Elemen	Implementasi
Heuristic	Tembak HANYA jika $ \text{gunBearing} \leq 3$ derajat ke arah musuh
Fungsi Seleksi	$\text{if } (\text{Math.Abs}(\text{gunBearing}) \leq 3 \ \&\& \ \text{GunHeat} == 0) \rightarrow \text{Fire}$
Gerakan	Wall hugging: menyusuri pinggir arena terus-menerus
Firepower	$\text{min}(3 - \text{gunBearing} , \text{Energy} - 0.1) \rightarrow$ makin akurat makin keras
Keunggulan	Hit rate tertinggi, setiap peluru hampir pasti kena, Survival tinggi
Kelemahan	Frekuensi tembakan lebih rendah karena menunggu akurasi sempurna

3.2.2 Bot Alt 1 - AggressiveHunter (Heuristic: Target Energi Terendah)

Strategi: Bot selalu memilih target dengan energi paling rendah di antara semua musuh yang terdeteksi, karena musuh dengan energi rendah

Elemen	Implementasi
Heuristic	Pilih musuh dengan nilai Energy minimum dari semua musuh terdeteksi
Fungsi Seleksi	$\text{if } (\text{energy} < \text{lockedEnergy}) \rightarrow$ ganti target
Gerakan	Membentuk angka 8 terus-menerus agar susah ditembak
Firepower	Maksimal saat musuh lemah, proporsional

	terhadap jarak
Keunggulan	Memaksimalkan Bullet Damage Bonus 20% dari kill musuh lemah
Kelemahan	Musuh terlemah belum tentu yang paling dekat, akurasi bisa rendah

3.2.3 Bot Alt 2 - ClosestTarget (Heuristic: Target Energi Terendah)

Strategi: Bot selalu mengejar dan menabrak musuh yang paling dekat karena memberikan kemungkinan hit tertinggi dan memaksimalkan Ram Damage Bonus sebesar 30%. Elemen

Elemen	Implementasi
Heuristic	Pilih musuh dengan nilai Distance minimum dari semua musuh terdeteksi
Fungsi Seleksi	if (dist < closestDist) -> ganti target
Gerakan	Langsung maju menuju musuh terdekat
Firepower	Tembak ringan (1.0) sambil mengejar, keras saat nabrak
Keunggulan	Memaksimalkan Ram Damage + Ram Bonus 30% dari kill via ramming
Kelemahan	Rentan kena peluru karena bergerak lurus menuju musuh

3.2.4 Bot Alt 3 - EnergyManager (Heuristic: Firepower Optimal)

Strategi: Bot selalu menghitung firepower yang paling efisien berdasarkan jarak ke musuh. Semakin dekat musuh, semakin besar firepower yang digunakan untuk memaksimalkan damage per energi yang dikeluarkan.

Elemen	Implementasi
Heuristic	Firepower = $\min(3.0, 500/\text{distance})$ -> optimal per jarak
Fungsi Seleksi	Hitung firepower optimal setiap kali musuh

	terdeteksi
Gerakan	Zigzag terus agar susah ditembak musuh
Firepower	Dinamis: 0.1 - 3.0 tergantung jarak ke musuh
Keunggulan	Efisiensi energi terbaik, tidak boros untuk tembakan jarak jauh
Kelemahan	Tidak memprioritaskan target tertentu, bisa tidak fokus

3.3 Analisis Efisiensi dan Efektivitas

Bot	Heuristic	Target Skor utama	Efisiensi	kompleksitas
Main Bot (UTAMA) Max Accuracy	Max Accuracy (bearing < 3°)	Bullet Damage + Survival	Sangat Tinggi	O(1) per turn
AggressiveHunter	Min Energy Target	Bullet Bonus 20%	Sedang	O(n) per turn
ClosestTarget	Min Distance	Ram Bonus 30%	Rendah	O(n) per turn
EnergyManager	Optimal Firepower	Bullet Damage	Tinggi	O(1) per turn

3.4 Strategi Greedy yang Dipilih

- Memaksimalkan hit rate dengan hanya menembak saat gun bearing kurang dari 3 derajat, sehingga hampir setiap peluru mengenai target.
- Gerakan wall hugging membuat bot selalu bergerak di pinggir arena, mengurangi kemungkinan diserang dari banyak arah.
- Survival rate tinggi karena posisi di pinggir arena membatasi sudut serangan musuh.
- Terbukti menang dalam pengujian melawan berbagai bot sample seperti RamFire, Fire, dan Walls.

BAB 4: Implementasi dan Pengujian

4.1 Implementasi Bot Utama - Main Bot

4.1.1 Pseudocode

```
moveAmount <- max(ArenaWidth, ArenaHeight)
peek      <- false
wallDir   <- 1

PROSEDUR Run():
// Posisikan bot di pinggir arena dulu
TurnRight(Direction mod 90)
Forward(moveAmount)
peek <- true
TurnGunRight(90)
TurnRight(90)

WHILE IsRunning DO
    peek <- true
    Forward(moveAmount) // Jalan menyusuri dinding
    peek <- false
    TurnRight(90 * wallDir) // Belok di sudut
ENDWHILE

PROSEDUR OnScannedBot(e):
dist <- DistanceTo(e.X, e.Y)

// GREEDY - Fungsi Seleksi:
// Tembak HANYA kalau gun sudah benar-benar mengarah ke musuh
// bearing < 3 derajat = hit probability tertinggi
gunBearing <- GunBearingTo(e.X, e.Y)
TurnGunLeft(gunBearing)

// Firepower: makin dekat makin keras, makin akurat makin keras
fp <- min(3 - |gunBearing|, Energy - 0.1)
fp <- max(0.1, fp)

IF |gunBearing| <= 3 AND GunHeat = 0 THEN
    Fire(fp) // Tembak dengan firepower optimal
ENDIF

IF peek THEN Rescan() ENDIF // Scan ulang target
```

```

PROSEDUR OnHitByBullet(e):
wallDir <- wallDir * -1      // Balik arah di dinding

PROSEDUR OnHitWall(e):
Back(30)                    // Mundur sedikit
TurnRight(90 * wallDir)     // Belok keluar dinding
peek <- false               // Reset scan flag

PROSEDUR OnHitBot(e):
bearing <- BearingTo(e.X, e.Y)
IF bearing > -90 AND bearing < 90 THEN Back(80)
ELSE Forward(80) ENDIF

```

4.1.2 Struktur Data dan Fungsi

Nama	Tipe	Fungsi
moveAmount	double	Jarak maksimum yang ditempuh saat menyusuri dinding (= max(ArenaWidth, ArenaHeight))
peek	bool	Flag untuk mengaktifkan Rescan() setelah tembak agar radar tidak kehilangan target
wallDir	int	Arah belok di sudut arena: 1 = searah jarum jam, -1 = berlawanan
Run()	Override	Loop utama: posisikan di dinding lalu jalan terus menyusuri pinggir arena
OnScannedBot()	Event	Inti logika greedy: arahkan gun dan tembak hanya jika bearing < 3 derajat
OnHitByBullet()	Event	Membalik arah wallDir saat kena peluru agar pola tidak tertebak
OnHitWall()	Event	Mundur sedikit dan belok agar tidak stuck di dinding

OnHitBot()	Event	Mundur atau maju tergantung posisi bot yang ditabrak
------------	-------	--

4.2 Pengujian

Pengujian dilakukan sebanyak 3 kali dalam format 1 vs 1 antara Main Bot (bot utama) dengan bot asisten. Setiap pengujian terdiri dari 10 rounds.

4.2.1 Pengujian 1 - Main Bot vs RamFire

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	TibaTibaJadiKelompok...	1154	300	60	604	114	76	0	6	2	0
2	Ram Fire 1.0	534	100	20	228	9	145	32	2	6	0

Analisis Pengujian 1: Main Bot menang telak dengan skor 1154 vs 534. Bullet Damage sebesar 604 menunjukkan strategi greedy akurasi tinggi (bearing < 3 derajat) bekerja dengan baik. Bullet Bonus 114 membuktikan bot berhasil membunuh musuh dengan peluru. Survival 300 menunjukkan bot bertahan cukup lama meski RamFire agresif menabrak.

4.2.2 Pengujian 2 - Main Bot vs Fire

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	TibaTibaJadiKelompok...	321	100	20	177	24	0	0	2	1	0
2	Fire 1.0	116	50	10	56	0	0	0	1	2	0

Analisis Pengujian 2: Main Bot menang dengan skor 321 vs 116. Bullet Damage Main Bot (177) jauh lebih tinggi dari Fire (56), membuktikan heuristic akurasi tinggi menghasilkan damage yang lebih konsisten. Main bot memiliki poin survival yang lebih tinggi (100) dibandingkan Fire yang hanya 50.

4.2.3 Pengujian 3 - Main Bot vs Walls

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	TibaTibaJadiKelompok...	285	150	30	93	2	10	0	3	2	0
2	Walls 1.0	197	50	10	130	0	7	0	2	3	0

Analisis Pengujian 3: Main Bot menang tipis dengan skor 285 vs 197 melawan Walls yang merupakan bot terkuat dari sample bots. Menariknya, Walls memiliki Bullet Damage lebih tinggi (130 vs 93), namun Main Bot unggul di Survival (150 vs 50) dan jumlah kemenangan ronde. Ini membuktikan strategi wall hugging + tembak presisi efektif untuk survival jangka panjang.

4.3 Analisis Hasil Pengujian

Aspek	Hasil	Kesimpulan
Win Rate	3 dari 3 pengujian menang	Strategi greedy akurasi tinggi terbukti efektif
Bullet Damage	604, 177, 93 poin	Tinggi saat musuh agresif, turun saat vs bot sejenis (wall hugger)
Bullet Bonus	114, 24, 2 poin	Berhasil kill dengan peluru pada musuh yang bergerak lebih terduga
Survival	300, 100, 150 poin	Konsisten bertahan berkat posisi di pinggir arena
Kelemahan	vs Walls (sesama wall hugger)	Bullet Damage rendah karena pola gerakan serupa membuat prediksi sulit

Secara keseluruhan, strategi greedy Main Bot berhasil mendapatkan nilai optimal pada skenario melawan bot agresif (RamFire, Fire). Namun pada kondisi melawan bot dengan strategi serupa (Walls), performa Bullet Damage menurun karena pola gerakan yang mirip membuat tembakan presisi lebih sulit. Strategi greedy tidak selalu optimal secara global, namun terbukti konsisten menghasilkan skor lebih tinggi dari lawan.

BAB 5: Kesimpulan dan saran

5.1 Kesimpulan

Penelitian ini berhasil menerapkan algoritma Greedy pada empat bot Robocode Tank Royale, di mana masing-masing bot menggunakan fokus strategi yang berbeda, yaitu akurasi maksimum (Main Bot), hemat energi (AggressiveHunter), jarak terdekat (ClosestTarget), dan kekuatan tembakan optimal (EnergyManager). bot Main Bot dipilih sebagai bot utama dengan strategi khusus, yaitu hanya akan menembak ketika posisi senjata sudah benar-benar lurus membidik musuh dengan tingkat kemiringan kurang dari 3 derajat. Hasilnya, bot utama ini terbukti selalu menang dalam semua uji coba satu lawan satu melawan bot asisten seperti RamFire, Fire, dan Walls. Keberhasilan ini didukung oleh strategi bergerak menyusuri dinding pembatas arena yang membuat bot terus bergerak sekaligus mempersempit ruang tembak bagi musuh, sehingga bot memiliki tingkat pertahanan yang sangat tinggi. Selain itu, pengaturan radar yang otomatis mencari ulang sasaran setelah menembak memastikan pergerakan musuh tetap terpantau dengan baik meskipun bot sedang berjalan di sepanjang dinding. Walaupun algoritma Greedy ini tidak selalu memberikan hasil yang sempurna dalam segala situasi terutama karena adanya penurunan daya rusak tembakan saat melawan bot yang memiliki strategi serupa sistem ini terbukti tetap konsisten menghasilkan total poin kemenangan yang paling tinggi.

5.2 Saran

Untuk meningkatkan performa bot lebih lanjut, ada beberapa langkah pengembangan yang dapat diterapkan. Pertama, sistem dapat dilengkapi dengan fitur prediksi posisi musuh untuk memperkirakan di mana musuh berada saat peluru sampai, sehingga akurasi tembakan bisa jauh lebih tinggi. Kedua, bot perlu ditambahkan sistem adaptif agar bisa mengubah strategi secara otomatis sesuai situasi pertempuran, misalnya langsung berubah menjadi sangat agresif ketika mendeteksi sisa nyawa musuh sudah hampir habis. Selain itu, batas sudut bidik senjata yang sebelumnya dipatok tetap sebesar 3 derajat dapat diubah menjadi dinamis berdasarkan jarak; hal ini karena musuh yang berada dalam jarak dekat sebenarnya lebih mudah ditembak walaupun sudut bidiknya sedikit melebar. Terakhir, bot juga bisa dikembangkan agar mampu mengenali pola pergerakan musuh, sehingga peluang peluru mengenai sasaran akan meningkat drastis saat menghadapi musuh yang bergerak dengan pola yang teratur atau berulang.

LAMPIRAN

Link Repository Github:

https://github.com/virst132109-glitch/Tubes_TibaTibaJadiKelompok.git

Link Youtube:

https://youtu.be/8Q0_YLEXRKE?si=jf_FfFlpJNu4Oyxc

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.	✓	
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.	✓	
3	Membuat laporan sesuai dengan spesifikasi.	✓	
4	Membuat video bonus dan diunggah pada Youtube.	✓	

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Munir, R. (2020). Strategi Algoritma. Informatika Bandung.

Nelson, M. M. (2001). Robocode. IBM DeveloperWorks.